OCP GLOBAL SUMMIT 2026

传统 BGP 之外：为超大规模 AI 集群解耦路由分发

Eddie Ruan（资深主任工程师、网络系统软件总监）· Dennis Cai（阿里云）

时长 24 分 21 秒

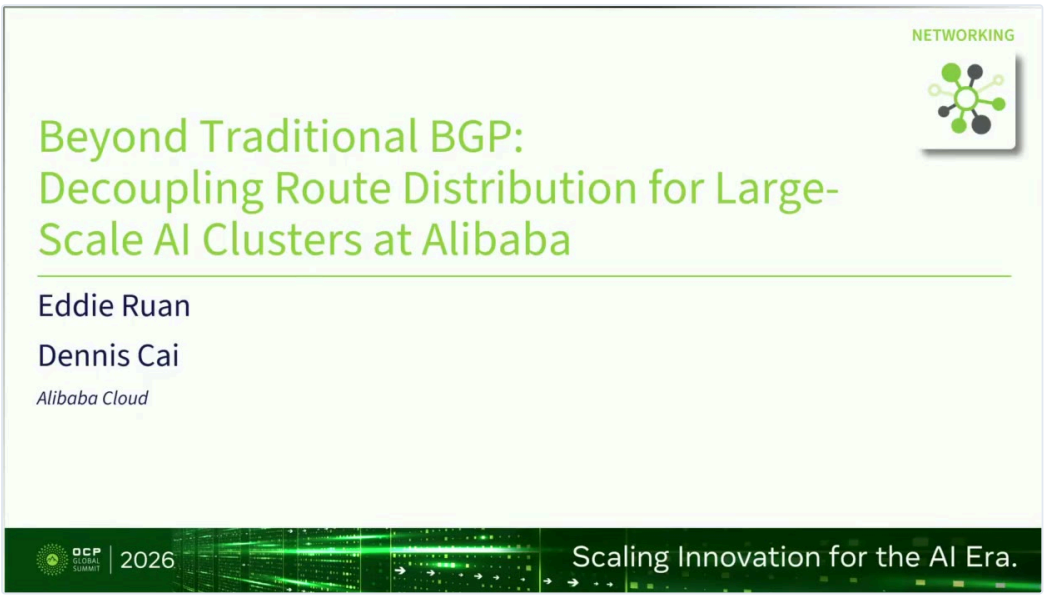
来源 youtube.com/watch?v=8in3TD9vW6g

页数 10 页幻灯片

传统 BGP 之外：为超大规模 AI 集群解耦路由分发

> OCP Global Summit 2026 · 网络 / AI 集群路由分会场 讲者：Eddie Ruan（资深主任工程师、网络系统软件总监）· Dennis Cai（阿里云） 全长 24 分 21 秒 ｜ 共 10 页幻灯片

01 开场：不用 MRC，怎么建 10 万 GPU 集群



标题页：Beyond Traditional BGP: Decoupling Route Distribution for Large-Scale AI Clusters at Alibaba，讲者 Eddie Ruan 与 Dennis Cai（阿里云）

好。呃，大家好。我是 Eddie，来自阿里巴巴团队。呃，我想今天是 OCP 的最后一天。呃，我不知道大家有没有跟那些一年才见一次的人好好聊聊。但今年——可能这一周我们已经聊过很多次，我已经见到很多老面孔了。

所以我大概给你们讲个故事看看，因为这一切都是 OCP，我们更像是把它当成一个"老男孩俱乐部"。我知道这话可能冒犯在座的女士，但反正你们也不会想加入这个老俱乐部。

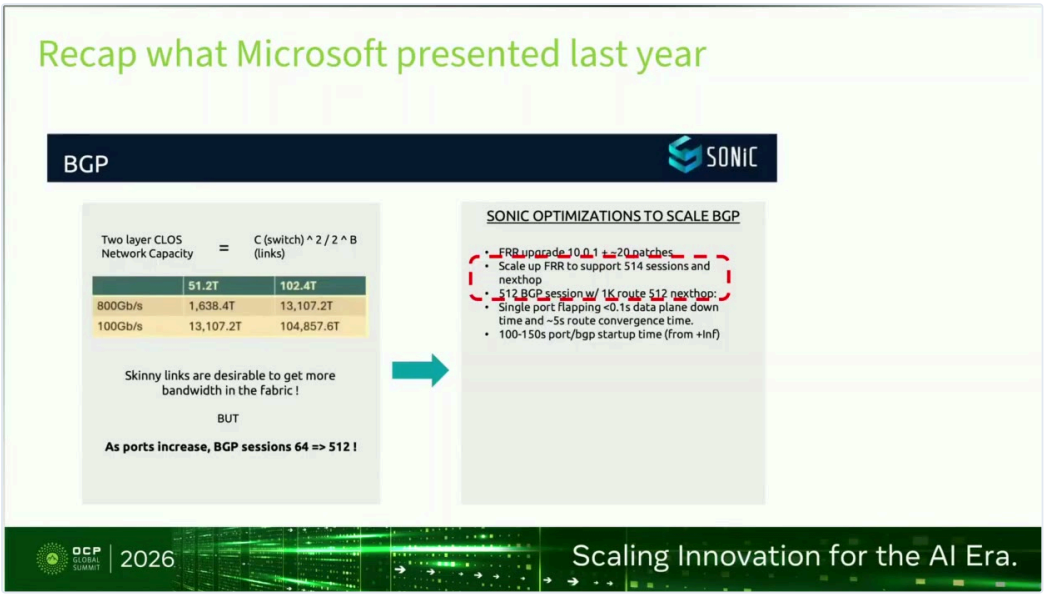
所以，基本上我想给你们展示的，就像我说的，这个挑战是怎么进入画面的，以及最终我们怎么在云上拿出一些好的方案。所以今天我想讲的是"遗产"——我们怎么在不使用 MRC 的情况下建一个 10 万 GPU 的集群。今天我们有很多关于 MRC 的讨论。所以我说我不讲，但我还是从 MRC 这一页开始。我们回顾一下，就像我说的，这是 Guhan 去年的幻灯片。他们做得非常出色。他们交付了——微软交付了大约 10 万个 GPU，连到 Fairwater 里……

> "Yeah. So, hi everyone. So, I'm Eddie. So, I'm from Alibaba team. So, I think today is the last day for the OCP. So, I don't know everyone have a good conversation for the people who you only meet once a year. But, in this year probably this week we did I see a lot of old face in multiple times already. So, I probably give you a story to see because the all this is OCP and the we have a more considered as a old boy club. I know if I offend any to the ladies here, but but you don't want to join this old club anyway. So, basically I want to show you like I say how this chat will come into the

> picture and eventually we can come up with some good solutions in the cloud. So, today I basically want to talk about the legacy how we could build a 100k GPU cluster without MRC. We today we have a lot of two MRC today. So, I would say I won't but I start with the MRC slide. Let's recap like I say that's just Guhan's slide from last year. Say they did an excellent job. They delivered Microsoft delivered like 100k the GPUs connecting in the"

小结　命题被限定得很窄：不是"怎么把 BGP 调优"，而是"在不改动主机、不引入 MRC 的前提下，让 10 万 GPU 的集群跑起来"。这个约束条件决定了后面所有设计——凡是要求租户配合的方案，在公有云里都不成立。

02　微软的答案：SRv6 把租户地址藏起来



Recap what Microsoft presented last year：左侧为 Fairwater 集群示意，右侧 SRv6 OPTIMIZATION AT SCALE 说明 500 个 BGP 会话、1K 条路由；红框标出 SRv6 encap 位于主机侧

……Fairwater 里。这一页我稍微标了一下。你看他们声称自己在改进 FRR，也就是 BGP 协议栈。但只有 500 个 BGP 会话、1K 条路由。可是如果你想支持 10 万个 GPU——那意味着 10 万个地址需要接入。所以 MRC 基本上用了一个很巧的办法回避了这件事，核心就是：我做 SRv6 封装。这样我就把所有租户或 GPU 地址从 fabric 里藏起来了。于是 fabric 里只有基础设施路由，非常小，可以做到 1000 条以下。这就是 MRC 漂亮的地方。

那么最初，像 Clarence……我不知道 Clarence 为什么把我标成 SRv6，这个我待会儿解释。但从阿里的角度看，我们已经讲过很多次，甚至今天 Trevor 也在讲我们用 SRv6 做 DCI。所以当我们听说微软用 SRv6 做 DCI 时，我们说"我的梦想成真了"，因为我们一直在想数据中心到数据中心、端到端地做 SRv6。我当时觉得那可能就是其中一个范例。因为 SRv6 最初起步的时候，我记得是 NTT 或者 KDDI 的人说"嘿，我们想从天线一直做到数据中心"。那是另一个梦想。

但呃，当我回去找我们内部团队，看我们怎么处理、能不能做 SRv6，或者直接把微软已经展示的作业抄一遍？答案是：不行，这事不会发生。原因是微软非常幸运。他们只有一个大客户——OpenAI，可以统一要求。所以极其幸运。而对我来说，因为我们是公有云，有海量客户。有些客户甚至说"我只要阿里云的一台裸

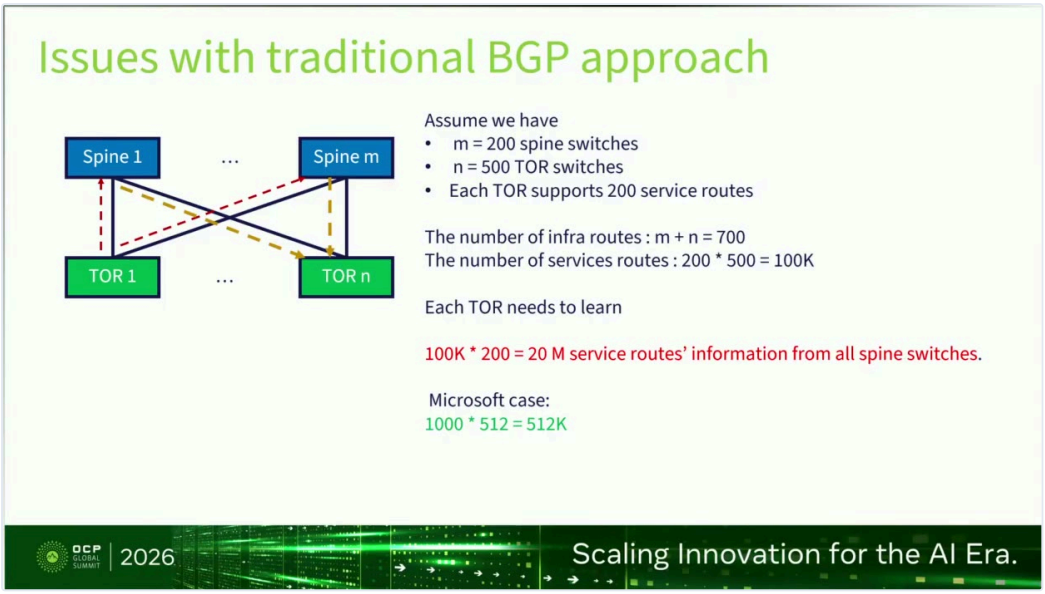
金属机器，上面不要任何阿里的硬件"。那我要怎么说服所有人都接受 MRC？这一点我连想都不太想。也许以后靠教育会改变，但目前不会发生。

所以基本上，我没法把微软的作业抄一遍。这就是为什么我说，今天我先给你们放个假——我今天不做 MRC，因为那太重了。今天我讲的更多是：我用不了 SRv6，那我们是怎么做的？

> "the fair water. And in this slide I highlight a little bit here. You see that they claim they is improving the FRR which is the BGP stack. But, the 500 BGP sessions with only 1K routes. But, if you're looking on the if you want to support 100k GPUs which means 100k like addresses you need to connecting it. So, MRC basically did a one sneak way to avoiding the the job is basically say I do the SRv6 incap. So, I hiding all the tenant or GPU addresses out from the fabric. So, fabric is only have the infra route. It's very small. So, you can do the less than 1,000. So, that's the beautiful the MRC. So, initially like Clarence, so I don't know why Clarence put me as a SRv6. I will explain this later, but you see that from the Alibaba point of view, we already talked multiple times even at today when Trevor talking about we doing the DCI use the SRv6. So, when we heard of Microsoft use the SRv6 doing the DCI and we say, "My dream is to come true because we always thinking about data center to data center end to end to do the SRv6." I was thinking that could be the one of the example for that. Because when SRv6 initially started, like I think that's um NTT guys or the KDDI guys say, "Hey, we want to do the from the antenna all the way to the data center." So, that's another dream. But uh when I go back to our like internal team to see how we handling it, can we do the SRv6 or copy the homework which Microsoft already showing up? And the answer is no. It won't happen. The reason is that like Microsoft is very lucky. They have only one big customer, the Open AI which can handle in it. So, it's extremely lucky. And for me, I have a huge amount of because we are public cloud, we have huge amount of customers. Some customers even say, "I only need a bare metal machine from the Alibaba Cloud, but I don't need any as Alibaba hardware on the top of it." So, how do I I won't be able to persuade everybody to take the MRC. That's one I'm not even want to think about it. Maybe education will later on what happens, but at this moment it won't happen. So basically say, I won't be able to cut make it a copy for the Microsoft the homework. So that's why I would say I will give you a break to say I found MRC today because it's a way too much. But today I actually I was in the my talk is more like say I won't be able to use the SRV6. How we doing it?"

小结　SRv6 的价值不在协议本身，而在它把"10 万个租户地址"从 fabric 的控制平面里彻底移出——fabric 只剩 700 条基础设施路由。但它的成立前提是主机侧可控：微软能要求 OpenAI 打开 SRv6，公有云不能。这一条前提差异，把两家逼上了完全不同的架构路线。

03　规模账：2000 万条路径是怎么来的



Issues with traditional BGP approach：Spine 1 到 Spine m 与 TOR 1 到 TOR n 的全互联示意；右侧算式——m = 200 台 spine、n = 500 台 TOR、每台 TOR 承载 200 条服务路由；基础设施路由 m + n = 700；服务路由 200 × 500 = 100K；每台 TOR 需从所有 spine 学到 100K × 200 = 20M 条服务路由；微软情形为 1000 × 512 = 512K

那我们来看看我们面对的问题是什么。呃，我举个例子——就像我说的，这是一个典型的 spine-leaf 拓扑。我随手丢一些数字方便计算。假设这是 spine，我有 200 台 spine。然后每台 spine 连 500 台 TOR。每台 TOR 连 200 个 GPU。这是第一个。

那我们看基础设施路由。回环地址就是你的基础设施路由。这是你唯一需要的东西。所以基本上 200 + 500，很简单，就是 700 条。

然后我们谈服务路由。因为传统上所有服务路由都要穿过这个 fabric。这里有一个概念叫 BGP path。那我们看今天的服务路由是 200 × 500，也就是 10 万条。这是我故意造的数字。然后每台 TOR 都需要学到全部这些，因为我们假设不做任何路由聚合——为什么不能聚合我待会儿解释。那意味着每台 TOR 都要从 200 个对等体那里学这 10 万条路由。对吧?

我们先不谈 BGP 的复杂度。有意思的是，网络分会场开了一天半，今天只有我一个人在讲 BGP——所以你大概能看出这个俱乐部里有多"老派"。

那么这意味着我需要 10 万 × 200，也就是 2000 万条路径要处理。我们甚至还没谈 BGP 的复杂度。单是这 2000 万个状态要管理，就已经是噩梦了。所以问题就摆在那里。

然后我们看看微软是怎么处理的。这就是为什么当他们说"我在优化"的时候，我其实有点失望——因为他们并没有真正在优化，我的问题还在那儿。对他们来说，是 1000 × 512，也就是 51.2 万条 BGP 路径。和我这边比只是一个零头。所以这就是为什么我说，我没法抄微软。

我也已经说了，我们得自己解决它。它很难，但必须解决。因为我们的业务就是要支撑它——我们有点像微软，也需要把 10 万 GPU 的集群拉上线去产生收入。

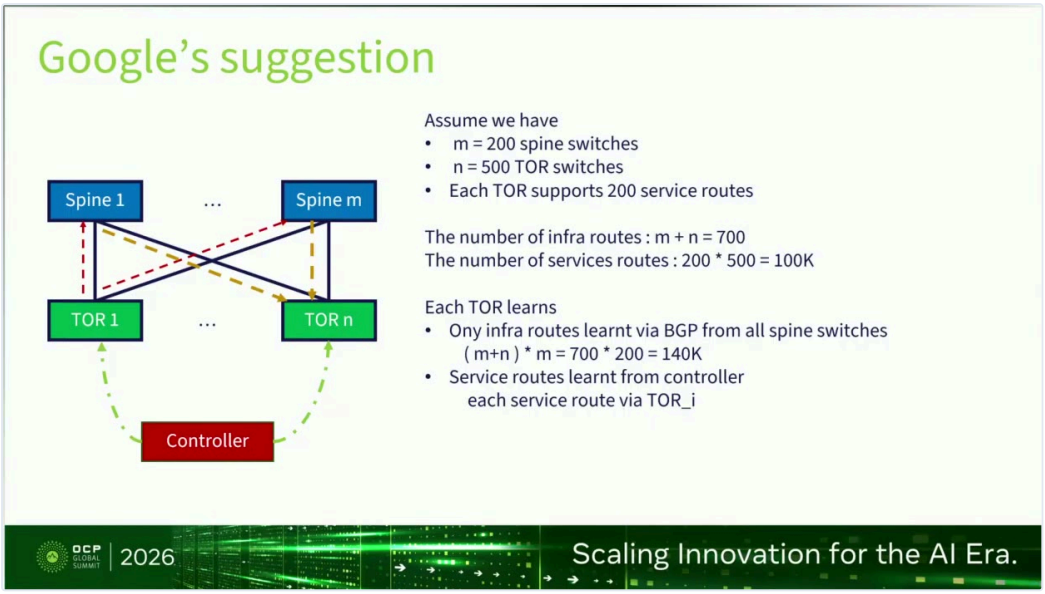
所以上一届 OCP 之后，我去找了 Google 的人聊。结果他们也有非常类似的问题。所以昨天有人告诉我他们在做 MRC，或者说他们要把我抛弃了……

> "So let's look at the what the problem what we have. So say if we want to then So I just like a story in some like I say this is a typical like a spine leaf topology. I just uh throw some numbers for easy calculation. So let's say it's spine. I have like 200 spines there. Then each spine will be connecting to 500 TOR switches. And each TOR will connecting to 200 like GPUs. So that's one. So let's look at the infra routes. The loopback addresses that will be your infra routes. You That's the only thing you needed. So then it's a basically say 200 + 500 is simple. It's just 700. Then we talking about like I say the service route. Because the traditionally all the service routes we carry through the this fabric. So we are talking about the one concept called BGP path. So let's see the service routes today is right now it's like 200 * 500 which is 100K. That's what just I I'm purposely making the the numbers. Then we need to each uh TOR need to learn all this one because let's assume we won't do the any aggregation for the routes. We I will later explain why I cannot do it. So then which means that for each TOR I need to learn that 100K routes from that 200 peers. Right? Let's not talking about all the BGP complexity. So it's a funny we are 1 and 1/2 day for the network sessions. I'm the only guy talking about the BGP today. So so you probably see the really really old library there in the club. So then which means that I need to have 100k time to 200, which is a 220 million paths we need to deal with. We are not even talking about the BGP complexity or these things. This one, 200 million state you need to deal managing, it's already a nightmare. So so you see that problem is sitting there. Then let's look at the what are the Microsoft the deal? They say I will That's why I will really be just disappointed when they say I'm optimizing it, but they are not really optimizing it because my problem is still there. For them, they are 1,000 time 512, which is a 512k those BGP paths. So it's a tiny fraction compared to what I have. So so that's why I like say I won't be able to copy the Microsoft. I also I already told you to say let's tackle it. It's hard, but we have to tackle it. So then so that's basically say we are it's hard, but we say but the our business is to have to to supporting it because we are kind of like the Microsoft. We also need to bring in that 1,000 100k BGP GPUs the cluster to be online to generate the revenue. So after the last OCP, I talked to the Google folks. It turned out that they are very good have similar problems. So that's why I like yesterday someone told me like they are doing the MRC or say they want to abandon me already."

小结　状态爆炸的算式是 100K × 200 = 20M，而微软是 1000 × 512 = 512K，差 39 倍。注意讲者口述为"220 million"，幻灯片标 20M——此处从幻灯片（100K × 200 本身也只能是 2000 万）。真正决定量级的不是路由条数，而是"每台 TOR 必须从每一个对等体各学一遍"这个乘法结构：只要服务路由还在 fabric 里，叶子节点的状态就随 spine 数量线性放大。

想法　这个乘法结构值得单独记一笔。10 万条服务路由本身不算多——一台现代交换机扛得住。真正压垮控制面的是"乘 200"：每台 TOR 要为每一条路由维护 200 份来自不同对等体的副本，因为 BGP 的选路需要知道每条路径是谁给的。所以后来所有方案（SRv6、控制器、RR）本质都在做同一件事——把"乘 200"里的那个 200 变成 4 甚至 1。降低路由条数反而次要。

04 两次"转身"：从控制器到 Route Reflector



Google's suggestion：左侧同样的 spine-leaf 拓扑（m = 200 spine、n = 500 TOR、每台 200 条服务路由），右侧说明基础设施路由 m + n = 700、服务路由 200 × 500 = 100K，红框内两条关键结论——只有与 MRC 相关的路由被通告和学习、服务路由不从 BGP 通告与学习

……但那也有点像，因为他们也有很多客户。也许 OpenAI 是其中之一，但他们也要面对公有云，有很多类似情况的客户。所以我说他们也有类似的问题。

但如果你把阿里和 Google 放在一起看，会很有意思。Google 是集中式 SDN 路线，我们在 BGP 这边是完全分布式的。在这个行业里，这是两个极端。所以当我们看到这个问题时，他们有 Virgo，我们有 10 万条 BGP 路由——就像我说的，总觉得对面的草更绿。于是我们开始聊。

去年 11 月，Bergado 基本上给了我一根救命绳。他给了一个很有意思的想法：为什么不干脆加一个控制器？把那 10 万条服务路由全部丢给控制器去下发。因为 GPU 不像 VM，GPU 根本不会动。它是物理待在那里的，跟 top-of-rack 绑死。所以 GPU 地址到 TOR 的绑定非常静态。那就可以用控制器去下发。这样一来，BGP 这边就非常简单了，只需要处理递归的基础设施路由。你不用再承载所有服务路由。10 万条服务路由全没了。

我喜欢这个想法，因为相比在 BGP 这边跟 2000 万条搏斗——那几乎是另一个"不可能的任务"——这个看起来非常有希望。因为我把东西放在控制器里，从难度上讲，我觉得对软件侧很多人来说，这是可做的。这给了我一个信号："嘿，不错，我们能做成。"

然后去年 12 月，我带着同样的想法去找 Clarence，说我想加一个控制器来解决问题。你看，这就是老男孩俱乐部有意思的地方——这个圈子里没有秘密。你有个问题，四处问问就行。他看了一圈，然后说：你并没有控制器啊，为什么要付这个代价？为什么不用 route reflector?

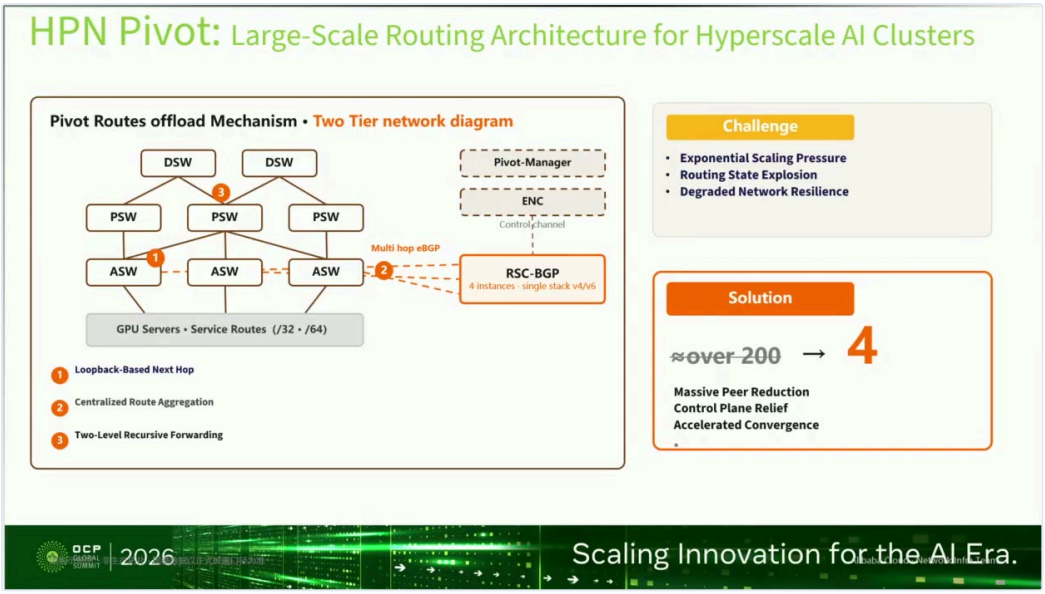
这是我们第一次考虑在云的数据中心里用 RR，因为我们以前从来没在那里引入过 RR。但这一次看起来是合理的。思路是：你不用在紫色那个位置放东西了。不用控制器，我用 route reflector。你的 TOR 去跟 RR 说话，RR 把路由发给其他 TOR。服务路由是同一个概念，只是把控制器换成了 RR。

这就更漂亮了，因为我是个 SONiC 的人。从 SONiC 的角度看，我需要写的代码是零行。这太甜了——你想，一个项目跟你说"零行代码"，那是梦想，对吧?

"But it's kind of like because they also have multiple customers. So maybe like a open eye will be one of but they also need to deal with the public cloud. There are a lot of customers also similar one. That's why I say they also have the similar problems. But if you looking on these two Alibaba and the Google, it's very interesting. Google is a centralized SDN approach. We are fully distributed on the BGP side. This is like the in this industry, this is a two extreme side. Right? So when we see this problem, they have Virgo, we have a 100K and the BGP which we feel like I say the other side of glass is more green. So we can we had we started talking. And the last November when Bergado basically threw me a lifeline. He gave me the very interesting idea to say, let's why not just add the controller? Let's put all the service routes that are 100K just to throw the controller to pushing down. Because GPU will not move not like a VM. GPU will not move at all. It's it's a physical sitting there. So it's kind of anchored with top. So then which means that the GPU address to the top binding is very static. So then you can use the controller to pushing down. So that's his idea. Then from the BGP side, it's a very simple. It just need to handling the the recursive infra routes. You don't need to carry the all the service routes. 100K service routes all gone. I like it because the compared to doing the 20 million battling on the BGP side which almost like an another mission impossible. This one looks very promising. Right? Because I put at a controller then from difficulty point of view, I think the on and a lot of people from software side, you see I it's doable. So that's give me the sign to say, "Hey, it's good. We would be able to do it." Then I last December, so I talked to Celerance with the same idea to say, say I want I want to add a controller to solving my problems. You see that's why the old boy club is very interesting. There's no secret in this valley. You have a problem, you'll just ask around. So, he looking around and he say like you don't have controller. Why pay that cost? Why not just using router reflector? This is the first time we are thinking about using router reflector in the data center in the cloud because we never introducing the router reflector there. But this time seems like reasonable. The idea is like I say you just don't have the so the same location in the purple one. Instead of using controller, I use the router reflector. Your tour will talk to router reflector. Router reflector will be sending the all the routes to other tours. Your service route is the same concept. The only thing is like it's a replacing the the as a router reflector. So, that's even beautiful because I'm a Sonica guy. From Sonica point of view, zero line of code I need to put in. That's a sweet because you think about it when you have a project, you say zero line of code to you need add. It's a dream, right?"

小结　两次建议把方案从"加控制器"推向"加 RR"：Bergado 给的是"GPU 不动 → 地址绑定静态 → 可以集中下发"这个观察；Clarence 给的是"你已经有 RR 了，为什么还要造控制器"。前者是架构洞察，后者是成本洞察。最终方案之所以能被采纳，很大程度是因为它在 SONiC 上不需要改代码——这比性能收益更能决定一个项目能不能落地。

05　HPN Pivot：三层结构、RSC-BGP 与 200 → 4



HPN Pivot: Large-Scale Routing Architecture for Hyperscale AI Clusters：左侧 Two Tier network diagram——DSW 在顶、PSW 居中、ASW 在底，ASW 下挂 GPU Servers（Service Routes /32 · /64）；右侧虚线框为 Pivot-Manager 与 ENC，通过 control channel 连到 RSC-BGP（4 instances、single stack v4/v6），ASW 与 RSC-BGP 之间为 Multi hop eBGP。三个机制：① Loopback-Based Next Hop ② Centralized Route Aggregation ③ Two-Level Recursive Forwarding。Challenge：Exponential Scaling Pressure / Routing State Explosion / Degraded Network Resilience。Solution：≈over 200 → 4，Massive Peer Reduction / Control Plane Relief / Accelerated Convergence

那我们把这些拼起来。这就是我们命名为 HPN Pivot 的项目。"Pivot"这个名字反映了我们的思考过程：我们从 MRC 出发，然后想做 BGP 优化，接着走 Google 的路线，最后采纳了 Cisco 的建议。所以"Pivot"（转身）这个名字，完美地反映了这段旅程。

那么这就是我们的 fabric。ASW 基本上相当于微软视角里的 T0，A 是 T1。而 DSW 是第三层，也就是 super spine。我待会儿解释，这一层主要是为了救援用的。它通常只用于异常报文，这个我稍后解释。

然后看右边，有一个 RSC-BGP，也就是我的 route reflector。我们叫它"路由服务集群"。基本上我们在那里放了一堆 RR。从 TOR 的角度看，我们只是把路由发布给这些 RR。唯一的区别是，我们不用接口地址，而用 loopback 地址。我们做 loopback 通告，把 loopback 地址保留在那里。然后 RR 会把所有这些路由发给其他 TOR。同时我们做递归路由。

也就是说，现在你的 GPU 可达性不再由 fabric 决定，而是由 TOR 的可达性决定。因为它是物理设备，不再是 VM，你不会频繁改动它。所以我们可以依赖这个静态绑定。

那么你看到直接的收益是：我从 400 降到 4。即使我有 10 万条路由，我也不用从 200 个对等体学，只需要从 4 个学。所以数据本身从原来的 2000 万降到 40 万。对吧？这非常可做。我很有信心，因为我就是个做 BGP 的，而且这事很容易做。

那另一边，可能有人会问：你为什么不能做路由聚合？如果你能做聚合，问题也就解决了。这是微软的同事问我的。两个原因。

第一，我们允许客户搬。如果 GPU 宕了，你把它移到另一个机架，我们允许地址跟着搬。这意味着从客户角度，他们什么都不用做，搬过去就完事了。这给客户灵活性，给我们头疼。这是其一。

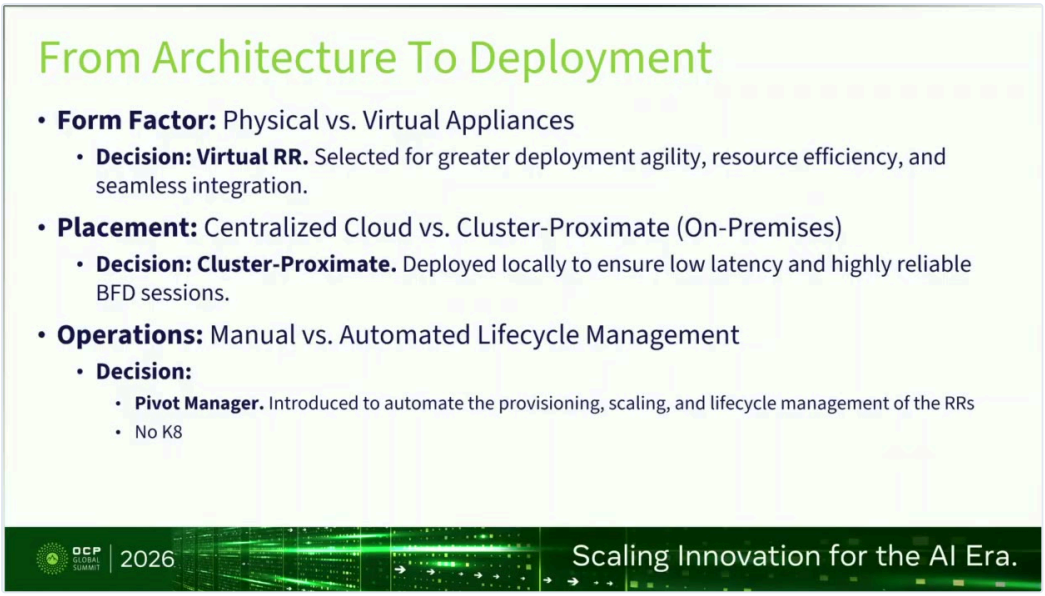
第二是 DSW 这一层。这基本上是我们的故障处理设计哲学。你是不是愿意相信你能在整站点上检测到一切？因为就在你检测到故障的那一刻，流量已经黑洞了。所以我们的想法是，特别是在发生问题时，我们用 DSW 跳到另一个平面。这是我们的信念。这样至少你能检测到延迟变高，但至少你不是唯一那个负责处理它的人。至少我们看到了故障，但不会变成灾难性故障。这就是我们的设计信念。所以这和微软以及其他人有点不同——他们一直是两层，我们是三层，但第三层主要用于异常处理。这是主要原因。

好，那么从架构角度看，它很干净，对吧？我零行代码，看起来我们把东西放进去就完事了。但当你深入到细节，从干净的架构到部署，那才是有意思的部分。我就列几个。

> "So, let's put it all these together. So, this is our which we name it called the HPM pivot for this a project. Pivot is basically you see that is an this name reflecting our thought process. We start MRC. Then we want to do the BGP optimization, then going to the Google's approach, then taking the finally taking the Cisco suggestion. Then we come that's why you see the pivot is really the perfect name to reflecting this journey. So, then this is basically say we have like I say two like I say the ASW is a basically the To from the Microsoft point of view and the A is the T1. So, this is the our like a fabric. And DSW is a third layer, which is a super spine. We I'll explain that this is the morning mainly for rescue purpose. So, it's normally one for only for exception packet, which I will explain later. Then let's look at it like say now on the right hand side there's a SRC the BGP, which means that this is a my route reflector. We call the route service cluster. Basically, we put a bunch of route reflector there. So, from the top point of view, we just publishing the routes to those route reflectors. Right? Then the only difference is like we are not using the it's own we use the loopback address. We are not using the interface address. So, we do the loopback appearing so sending the So, that it keeping that loopback address there. Then route reflector will be sending all these routes to the other tours. Then and also like we will do the recursive routing. So, which means that yeah, now for this one you are GPU reachability is not determined by the fabric. It's determined by the tours reachability. Because you are physical device, it's not a VM anymore. You won't be able to change that often. So, we can rely on the static binding there. So, that's the one. So, then you see that I immediate benefit will be I reducing from the 400 to four. So, even I have a 100k routes, you don't have to I only learn from the four four peoples instead of learning from 200 peoples. So, my the data itself will reduce to the 400k compare original 20 million. Right? This is a very doable. I'm very confident because I'm a BGP and also it's very easy to doing it. So, then other side the probably what people were asking say, why you cannot do the route aggregate, right? If you can do it because that's the what the some Microsoft folks ask me or say, if you can do the route aggregate, it's also the problem. Two reasons. One is we allow the customer to carry if the GPU will go down. You move into different rack. We allow the address to be moved around. Which means that the from the customer point of view, they don't need to do anything. They just bring that then everything is done. That give the flexibility to the customer, give a headache to us. That's one. The second one is the DSW layer. We somehow they basically this is a philosophy design philosophy for the failure handling. Do you want to believe everything you can detect on the whole site because even the moment you detect a fail, it's black hole in the traffic. So, we are thinking that we especially have happens, we use the DSW to jump into different plan. So, that's our belief. So, at least you can detecting the latency go high, but at least we won't you are not the single responsible guy to handling it. So, at least we see the outage, but we won't be the catastrophic outage. So, that's our design belief. So, that's a little bit different with Microsoft and the matter because they always two layer. We are third layer, but third layer is mainly for the exceptional handling. So, that's the the main reason. Okay. So, now from the architecture point of view, it's pretty clean, right? I have a zero code to be put in, right? And it seems like we just putting it this part should be simple. But when you go into the detail from the architecture, the clean architecture to the deployment, that's a of interesting part. So, I just"

> **小结**　方案由三个机制咬合而成：loopback 作为 next hop（让服务路由与接口解耦）、集中式路由聚合（在 RR 侧把 10 万条压成少量前缀）、两级递归转发（GPU 可达性由 TOR 可达性推出，而非由 fabric 逐条承载）。结果是对等体 200 → 4，路径 20M → 400K。注意讲者口述为 "HPM Pivot"，幻灯片写 "HPN Pivot"，此处从幻灯片。

06　从架构到部署：三个决策点



From Architecture To Deployment：Form Factor（Physical vs. Virtual Appliances）→ Decision: Virtual RR，selected for greater deployment agility, resource efficiency, and seamless integration；Placement（Centralized Cloud vs. Cluster-Proximate On-Premises）→ Decision: Cluster-Proximate，deployed locally to ensure low latency and highly reliable BFD sessions；Operations（Manual vs. Automated Lifecycle Management）→ Decision: Pivot Manager，introduced to automate the provisioning, scaling, and lifecycle management of the RRs，No K8

第一个是形态。你用物理机还是虚拟机？这没什么可犹豫的，因为我只用 FRR。所以我把 FRR 塞进 Docker，它就变成虚拟设备。理想情况下我应该用虚拟设备，用物理的没道理，因为我不需要任何数据面行为。

但有一个坑。我们是物理网络。就像我说的，我们所有的存量都是：把设备搬上机架、插线、上电。现在你有个虚拟设备，你拿它怎么办？这给我们带来很多头疼，因为作为物理网络团队，我们从来没这么干过。所以这是另一类问题。

第二个是部署位置。你把它放哪儿？可以放云上，也可以放在靠近集群的地方。我们选择靠近集群。原因是 BFD 保护——我们希望能够保护 BGP 会话，避免要等 90 秒超时。我们希望做到 1×3，甚至 300×3，那样更好。但如果你放云上，谁知道它在哪儿？可能一个在中国，一个在美国。所以我们希望就近。那么 BFD 就由你们来决定。

这意味着我们需要自己管理这些虚拟设备的生命周期，这就引入了运维复杂度。那怎么处理？你要处理：怎么把那些虚拟机拉起来，怎么监控这些服务器……所以我们引入了一个新的软件组件叫 Pivot Manager，专门做这件事的自动化。所以我在 SONiC 侧没让团队写一行代码，但我让他们去写 Pivot Manager。

这里公开的、有意思的部分是：我们总说要做产品化，因为我不想总安排人 on-call 去干这个。但如果你去问运维团队要需求，他们只给你很粗的几条，完全没有细节需求。原因是他们从来没做过——他们一直跟硬件机器打交道，从没跟软件打过交道。所以这是个挑战。现在还在进行中。

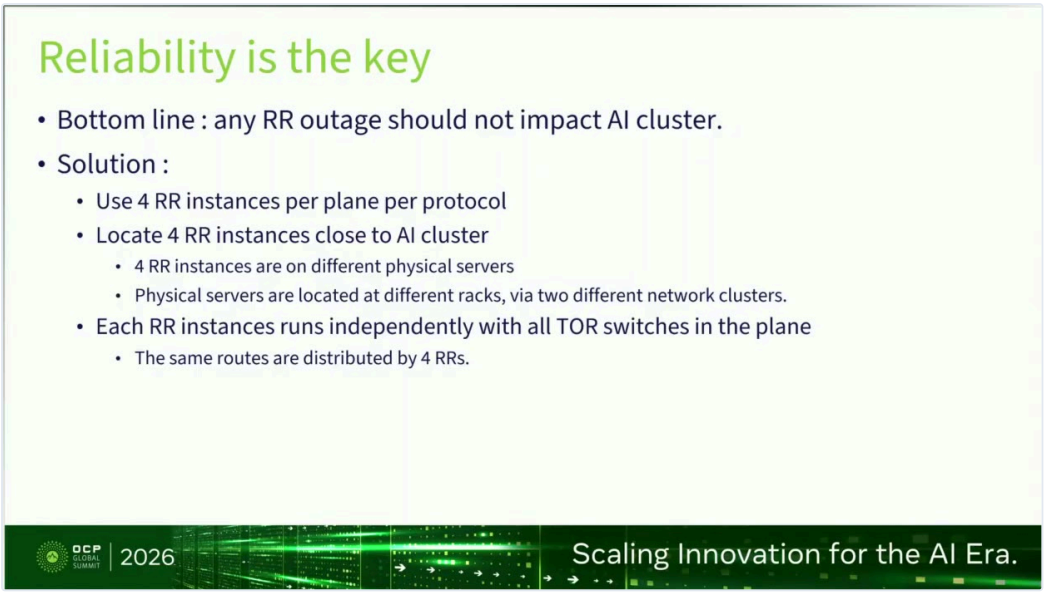
我这里没有用 Kubernetes。虽然我用 Docker，但 K8s 太重了。我手上就一把——几十个，20 到 30 个 Docker。所以我说不要 K8s，直接管它们，降低复杂度。

> "list a couple of them. The first one in the front factor. You using the physical machine or you using a virtual machine. Right? And that's no brainer because I'm only in the FRR. So, I shuffling FRR to be the the Docker, then it is a virtual devices. So, ideally I should use a virtual devices. It It doesn't make sense for me use the physical one because I don't need any data plan behaviors. But, there's one catch. We are physical networking. Right? All our, like I say, install base is like you put to the device on the rack, stacking that cable, power on. Right? Now, you have a virtual device. What do you do with that? So, that bring a lot of headache for us. So, because as a the physical network team, we never done it before. So, so different set of problem. The second one is the placement. Where you want to put it? We can put it in the cloud. Or we can put it close to where the cluster will be. And we choose the close to the cluster. The reason in the BFD protection. Because we want to be able to protect the BGP sessions. So, to avoiding that you need to wait for the 90 second time out. So, we want to be the one time three or even like a 300 times three, right? It's even better. But, if you want to put the cloud, who knows where is it? It might be we are sitting in the China, the other one in the in the US, right? So, we want to be close by. Then you guys determine the BFD. So, that means that we need to managing the life cycle of those virtual device on our own. So, that introducing the operation complexities. So, how do you deal with that? You need to deal with the now I set up the how do you bring up those virtual machines, how do you bring up the I'm monitoring these are servers, which So, we introducing that new software piece called the pivot manager, which is just do the automation for that. So, I didn't find my team on the Sonic side single line of code, but I found them to do the the pivot manager. And this is the public will be interesting part is like I say, if you talk we always say I want to be productionizing for this one because I don't want to always put someone to on call to doing it. So, but if you asking for the requirement from operation team, they only give you high level. There's no detail requirement at all. The reason is they never done it. They always deal with hardware machines. They never done with software. So, that's a challenge. So, now we are still ongoing basis. I put in the no kubernetes here. The although I'm using docker, kubernetes too heavy. I only have hands full like I say, like a couple tens of the 20 or 30 like dockers. So, that's why we say no kubernetes. Just handle them directly to reducing the complexity."

小结　"零行代码"的代价被摊到了运维侧：RR 虚拟化之后，团队第一次要自己管虚拟机的生命周期、监控、拉起与扩缩，于是多出一个 Pivot Manager。三个决策点里，只有第一个是纯技术的（不需要数据面，所以用虚机）；后两个都是被现实逼出来的——就近部署是为了 BFD 的毫秒级失效检测，不上 K8s 是因为规模只有二三十个容器，引入编排反而是负债。

想法　这段最值得抄的不是架构，是"运维团队给不出需求"这条观察。原文说得很直白：运维一直跟硬件打交道，从没管过软件，所以只能给你高层需求。这是所有"把网络功能软件化"的项目都会撞上的墙——技术方案再干净，验收标准仍然要靠自己定义。反过来说，这也解释了为什么很多团队宁可买笨重的物理设备：不是性能需要，是运维模型没准备好。

07 可靠性：四副本为什么会一起挂



Reliability is the key：Bottom line——any RR outage should not impact AI cluster；Solution 五条——Each RR instance per plane per protocol、Local RR instances close to AI cluster、RR instances on different physical servers、RR instances in different racks、Each RR instance runs independently with all TOR switches in the plane、The route model is distributed by BFD

另一个是，我们现在引入了 RR，它必须可靠。底线是：任何 RR 故障都不应该影响 AI 集群。因为你有 10 万 GPU，能用那 10 万 GPU 的人，通常可以直接给我们 CEO 打电话。大客户，对吧？你惹不起他们。所以必须处理好。可靠性非常关键。

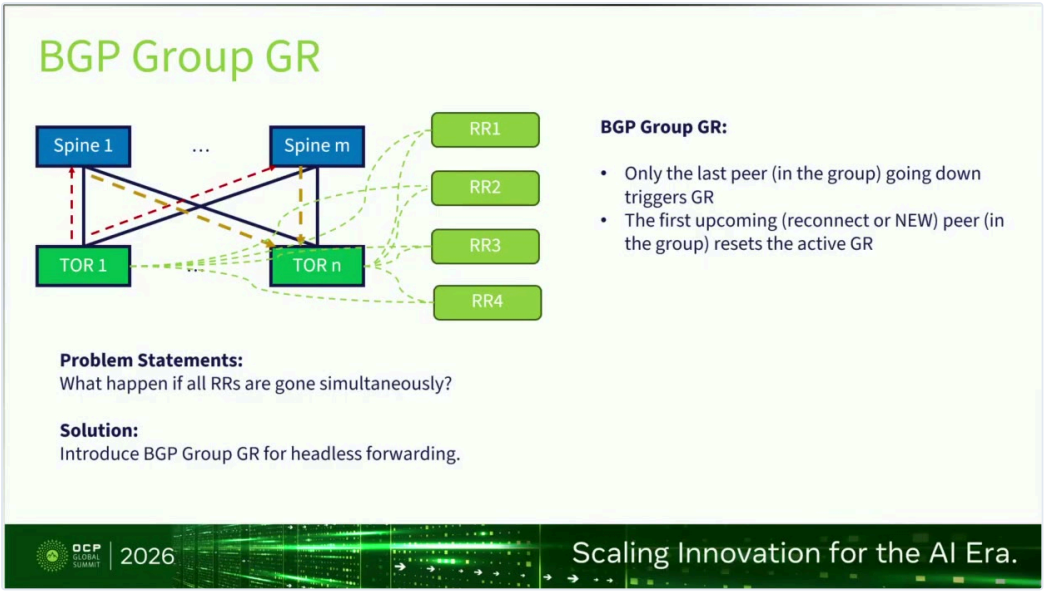
所以我们说：每个平面、每个协议放 4 个 RR 实例。我们甚至把 v4 和 v6 分开。我们就是这么做的。然后，既然我们把 RR 放到靠近 AI 集群的地方，这又引入了一组新问题——它在同一个园区里。所以我必须确保它们是物理服务器，4 个都在不同的服务器上，这是最低要求。然后它们在不同的机架下。然后我们甚至把它们放到不同的集群。然后每个 RR 各自独立运行。这是 RR 这一侧。

但这就够了吗？实际上不够。因为我们靠得近，在同一个园区。你可能遇到一栋楼断电——一场"完美风暴"，全没了。而且我们以前确实见过这类问题：在别的服务器上，差不多一小时内，大约 10 台正在跑的服务器静默重启了。

> "The other one is like now we introducing the R. Need to be reliable. So, the bottom line is like any R outage should not impact the AI cluster. Because you have the 100K uh GPU. Whoever be able to using that 100 GPU, normally they can make a phone call to our CEO. Big customer, right? They you cannot mess up with them. So, you have to deal with that. So, reliability is very crucial. So, we put we say, "Hey, we put a four R instance per plan per protocol." We even separate we 4 V6. So, we doing that. Then, since we put like a R into the close to the the AI cluster. Actually, that introducing a new set of problem, which means that uh it's in the same campus. Right? So, which means that I need to make sure they are physical servers, all four of them on different servers. That's minimum, right? Then they are under the different uh um rack. And then even we are moving them to a different uh cluster. So, so then then we also have running the the R each other run independently. So, that's the R one. But, is that good enough? And actually, no. Because we are closer by, so we in the same campus. You could have like I say one building outage, a perfect storm. Something gone. And also, we do see the issue before in the our other servers within like uh perfect 1 hour. About the 10 proving servers just to reboot silently."

> **小结**　四副本、跨服务器、跨机架、跨集群、独立运行——这套隔离做到了常规意义上的高可用，但没解决"相关性故障"：为了低延迟把 RR 放在同一园区，就等于把它们放进了同一个电力与制冷故障域。讲者举的例子很具体：一小时内约 10 台在跑的服务器静默重启。分布式冗余假设故障独立，而同园区恰恰破坏了这个假设。

08　Group GR：把"无头转发"搬进 BGP



BGP Group GR：左侧为 BGP Group GR 示意——仅当组内最后一个 peer 下线时触发 GR；右侧 BGP Group GR 三条——Only the last peer in the group going down triggers GR；That peer triggers GR, while the rest is doing GR or in the group, the whole group will be GR；Problem Statement: What happened if all 4 RRs are gone simultaneously? Solution: Introduce BGP Group GR for headless forwarding

这意味着有风险：我可能同时失去全部 4 个。这不像服务器供应商，他们的物理机分布在不同城市，所以没这个问题。但我们这里有。

所以这就是为什么我们引入——我们和 Cisco 合作——引入 group GR。我们想引入 graceful restart（优雅重启）。但 GR 通常是点对点的，你只能那样做。我们想做到组级别。我们不想搞乱 BGP 状态机，我们在上面加了自己的策略。意思是：如果 4 个全没了，我们才触发 GR。这样路由会被保持住。但如果其中任何一个回来了，它就恢复。所以这和现有的 GR 概念有一点不同，我们把它叫做 group GR。

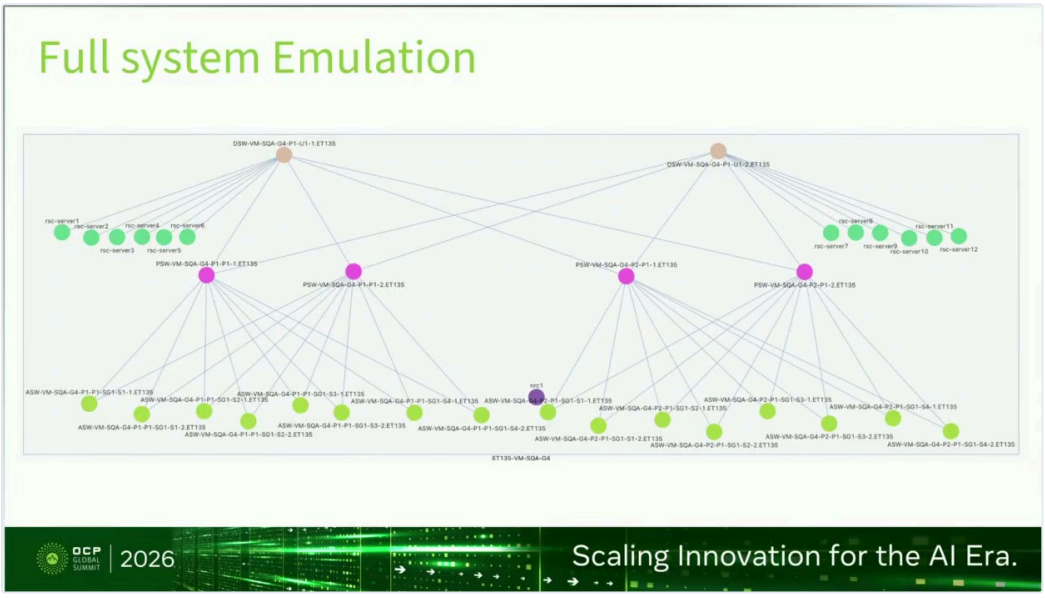
我们引入 group GR，还顺带解决了一件事——类似 headless forwarding（无头转发）。因为如果你用控制器，你天然就有 headless forwarding。Group BGP 没有。这基本上是把 headless forwarding 引入到 BGP 里。

> "So, which means it's will be have the risk. I could losing all four of them simultaneously. It's not like server provider, they are physical machines sitting on different cities. So, they don't have this problem. But, we do have this problem here now. So, that's why I introducing uh we working with Cisco for camera and the crease basically introducing a group GR. Basically, so we want to introducing a graceful restart. But, graceful restart normally is a peer to peer. You only do that. And we want to be the group level. So, we don't want to mess up the the BGP state machine. We put the our policy on top of it. Which means that if all four of them gone, then we trigger the GR. So, the that is the I'm to holding on the routes. But, if any one of them come back, he recover. So, that's a little bit of a from the existing the for

> GR concept, but that's what we call the group GR for that. So, we introduce the group GR and there to save this more like say headless forwarding because if you use controller, you natively have headless forwarding. Group BGP doesn't have that. This is basically introducing the headless forwarding in the BGP."

> **小结**　Group GR 是对"相关性故障"的补丁：把触发条件从"单个 peer 重启"改成"组内最后一个 peer 下线"，从而在四个 RR 同时消失时仍保持转发表不被撤回。它也补上了 BGP 相对控制器架构缺失的一项能力——无头转发。这是全文唯一一处明确承认"控制器方案本来更省事"的地方：控制器的无头转发是原生特性，BGP 这边得自己造。

09　全系统仿真：从设备级做到芯片级



Full system Emulation：网络拓扑仿真示意，浅绿色节点为 leaf 与 spine，左右两侧为承载 RR 的 VM；连线表示真实配置与真实连接

另一个是进一步测试。你有 10 万 GPU，你指望一切第一天就正常工作——那是个美好的愿望清单，不会发生。你必须测。

所以这是我们的仿真环境。因为——这是我同事明天在 SONiC 扩展工作坊也会讲的——我们基本上仿真整个系统，不只是设备级，我们做到芯片级仿真。所以除了数据面，其他一切都一样。是真实配置、真实形态，所有连接都一样。

如果你看到那些浅绿色的、紫色和粉色的点，那基本上是我们的 leaf 和 spine。左右两侧是跑在我们上面的 VM。所以我们可以仿真整个东西。甚至更多——因为我们在里面放了很多配置和监控工具。这就是为什么我叫它"全系统端到端仿真器"：我们不仅仿真设备或 RR，我们还让整个编排层能够在上面操作。

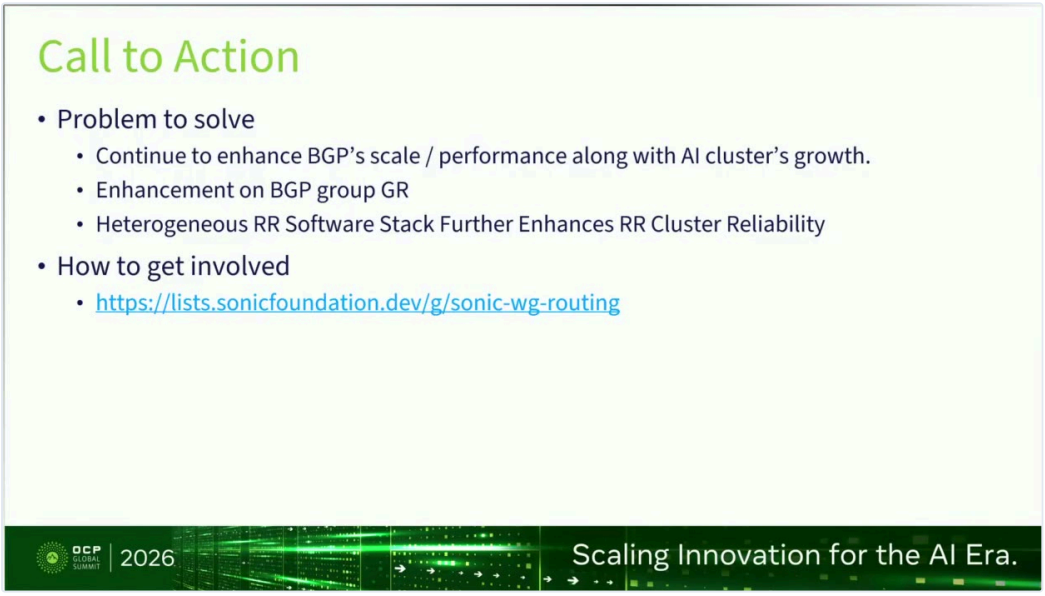
这降低了很多风险，因为一旦你测过一切，你只要在真实硬件上跑同样的东西——同样的配置、同样的连接——就不太可能出错。

> "The other one is the further testing. So, you have like I say 100k GPUs. You expecting everything will be day one action working. That's a good wish list. It won't happen. You have to test it. So, this is our emulation environment. When we

> see a system because this is one my colleague will talking in tomorrow's uh SONiC extended workshop also. Basically, we will simulate the whole system, not only the device device level, we do the chip level emulation. So, we get exactly the same one beside the data plane, everything will be the same. So, it's a real configuration, real like um form factor, everything connection will be the same. So, the if you see the light green one and the the the purple and the pink one dot, this is basically our two leaf and spine. Then the left and right side, this is our like a VM which are hosting on ours. So, we can basically simulate the whole thing. And even more because there are a lot of like configuration monitoring tools we are putting there. That's why I call the end full system end-to-end simulate emulator which we would not only simulate the device or the R, we also let the whole orchestration layer be able to hands on it. Then that reducing a lot of risk and because once you testing everything, you just run the same thing again in the real hardware which is same configuration, same connection. So, that won't be have a chance to have a mistake."

> **小结**　仿真的关键选择是"除数据面外全部真实"：芯片级仿真意味着配置、形态、连接与生产完全一致，于是同一套配置可以原样搬到真机上。这比"仿真器能跑通协议"更重要——它把"上线"变成"重放"，而不是"重新配置一次"。对 10 万 GPU 这种规模，一次配置错误的代价远高于搭仿真环境的成本。

10　100K 已上线，500K 在路上



Call to Action：Problem to solve 三条——Continue to enhance BGP scale / performance along with AI cluster's growth；Enhance BGP Group GR；Heterogeneous RR Software Stack Further Enhances RR Cluster Reliability。How to get involved：github.com/sonic-net 下的 routing 仓库

大概就是这样。今天我给你们讲的——这 10 万（集群）我们已经上线了。实际上已经有两个集群了，已经部署了。但随着 AI 往前走，我们不会原地不动。

呃，在最近的 S 大会上，阿里已经宣布了一件对我来说也很头疼的事：不再是 10 万，现在是 50 万。你怎么处理 50 万？所以我才需要老男孩俱乐部回来，我们讨论一下，再帮我一次。

所以接下来我们可能需要——这次我避开 BGP 增强，可能又得从那方面下手。Group GR 肯定是需要的。另外也许还有异构（heterogeneous）排序之类的。如果有人感兴趣，我们可以在 SONiC 路由工作组里讨论。我就讲到这里，谢谢大家。

> "So, that's uh pretty much I just give you So, today this 100K we already put online. So, actually there's two cluster already. So, it's already deployed. But, as AI moving, we won't be still stand still there. And uh in the recent uh S power conference, Alibaba already announced that one big headache also for me is like I said, it's no longer be 100K. It's a 500K now. How do you deal with the 500K? So, that's why we I need the old boy club come back. Let's discuss. Need my help again. So, we probably need to like I say, uh this time I avoiding the BGP in enhancement. I probably do attacking in that again. And the group GR definitely we needed. And also the maybe heterogeneous order something. So, if anyone be interested, we can discuss this uh Sonic routing working group. So, that's all I have. Thank you everyone."

> **小结**　落地状态是"两套集群、10 万 GPU 已上线"，而下一个数量级（50 万）已经在议程上。行动号召只有三件事：继续提升 BGP 的规模与性能、完善 Group GR、引入异构 RR 软件栈以进一步提升可靠性。三者都指向同一个方向——把控制面从"单厂商单实现"里解耦出来。

结语

一句话：**路由状态爆炸不是靠更快的 BGP 解决的，而是靠改变"谁需要知道什么"解决的。**

阿里的做法是把服务路由从 fabric 里摘出去，只留基础设施路由，再用 RR 把 200 个对等体收成 4 个。代价不是代码——SONiC 侧零行——而是运维：多了一套虚拟机的生命周期管理，多了一个 Pivot Manager，还多了一个"同园区四副本会一起挂"的漏洞要补。

这也是这篇演讲最有价值的地方：它没有把方案讲成一个干净的胜利，而是把三个决策点、两个补丁、一次没解决的相关性风险都摊开了。

数字速览

集群规模　10 万 GPU（已上线两套）→ 50 万在规划
拓扑假设　200 台 spine × 500 台 TOR × 200 GPU
基础设施路由　m + n = 700 条
服务路由　200 × 500 = 10 万条
改造前状态　10 万 × 200 = 2000 万条路径
微软对照　1000 × 512 = 51.2 万条
改造后　对等体 200 → 4，路径 2000 万 → 40 万
可靠性　每平面每协议 4 个 RR 实例，v4/v6 分离

本文依据 OCP Global Summit 2026 现场录像整理，数字以幻灯片标注为准；讲者口述与幻灯片不一致处（如 "220 million" 与 20M、"HPM" 与 "HPN"）已在对应小节注明。原片：youtube.com/watch?v=8in3TD9vW6g（24 分 21 秒）。如果你也在做超大规模集群的路由设计，欢迎留言聊聊你那边是选集中式控制器还是 RR——以及 50 万 GPU 这个量级，你觉得现有 BGP 还能撑住吗?

附录　时间轴

时间	内容
00:06	开场：不用 MRC 怎么建 10 万 GPU 集群
01:06	微软 Fairwater 的 SRv6 方案与它的前提
04:15	规模账：2000 万条路径的乘法结构
07:15	Google 的集中式路线；Bergado 与 Clarence 的两次建议
10:51	HPN Pivot 架构与 200 → 4
15:15	从架构到部署：虚机、就近部署、Pivot Manager
18:27	可靠性：四副本与同园区相关性故障
19:48	Group GR 与无头转发
21:24	全系统仿真
23:03	100K 已上线、500K 挑战与行动号召